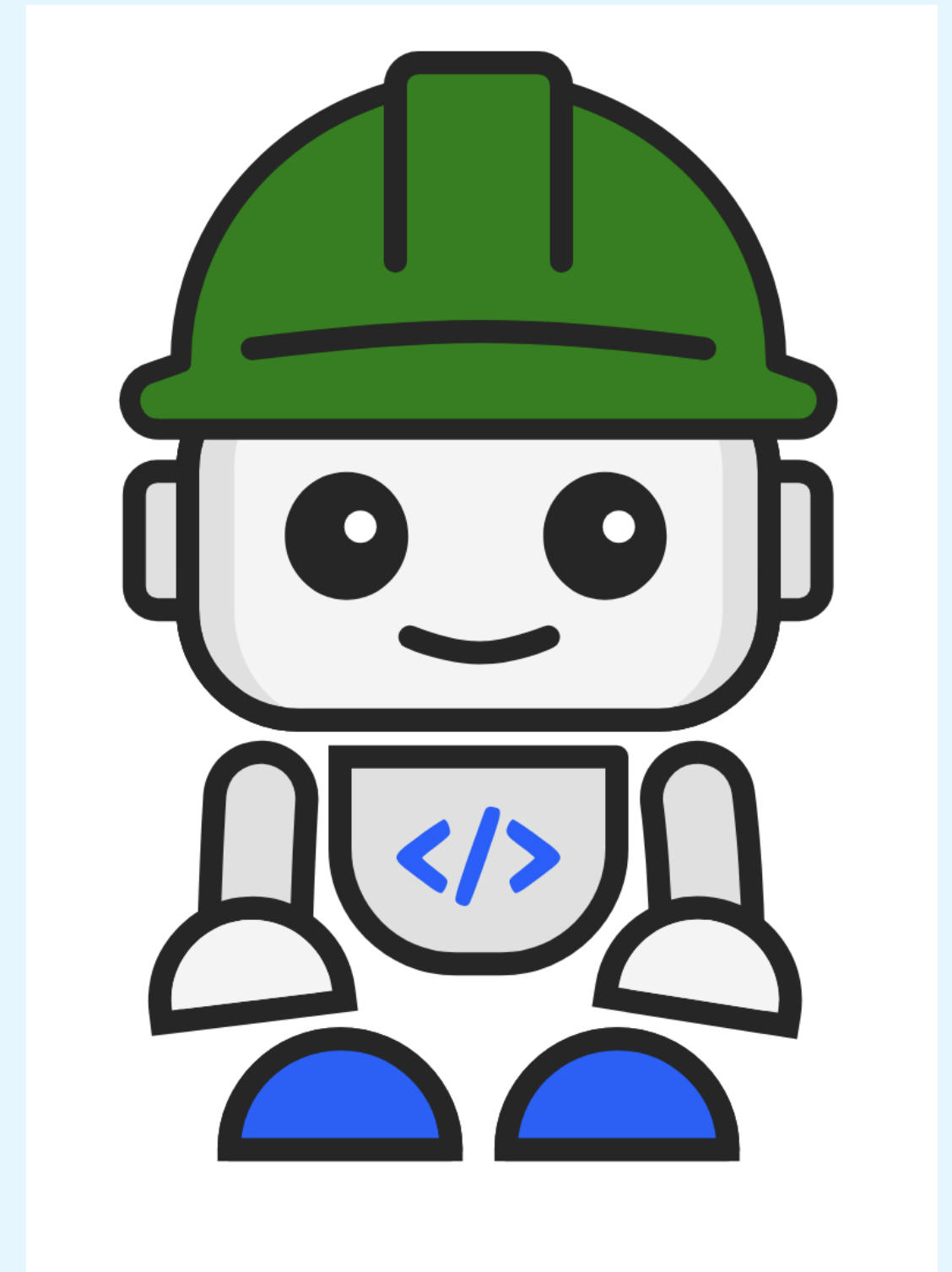
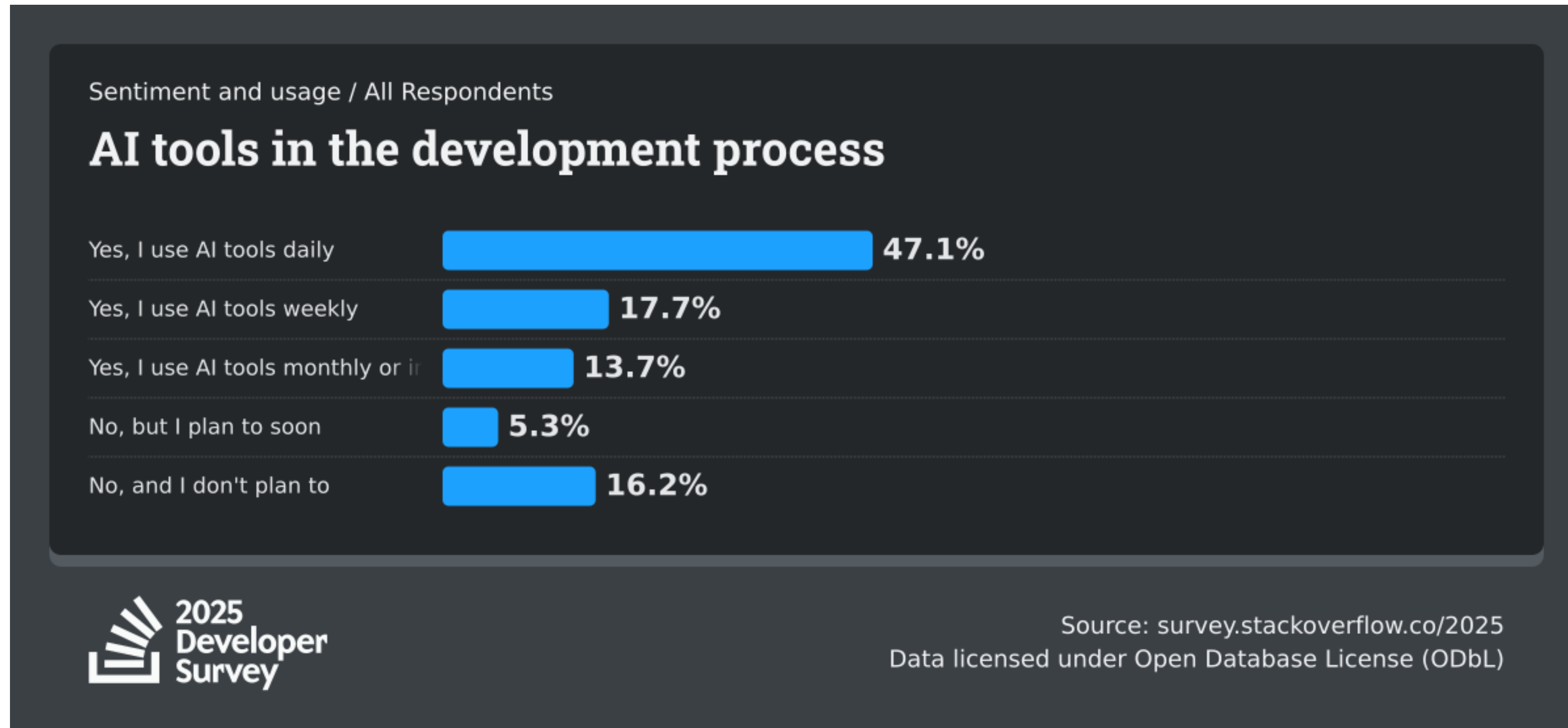


EcoCoder

The missing layer between infrastructure and code

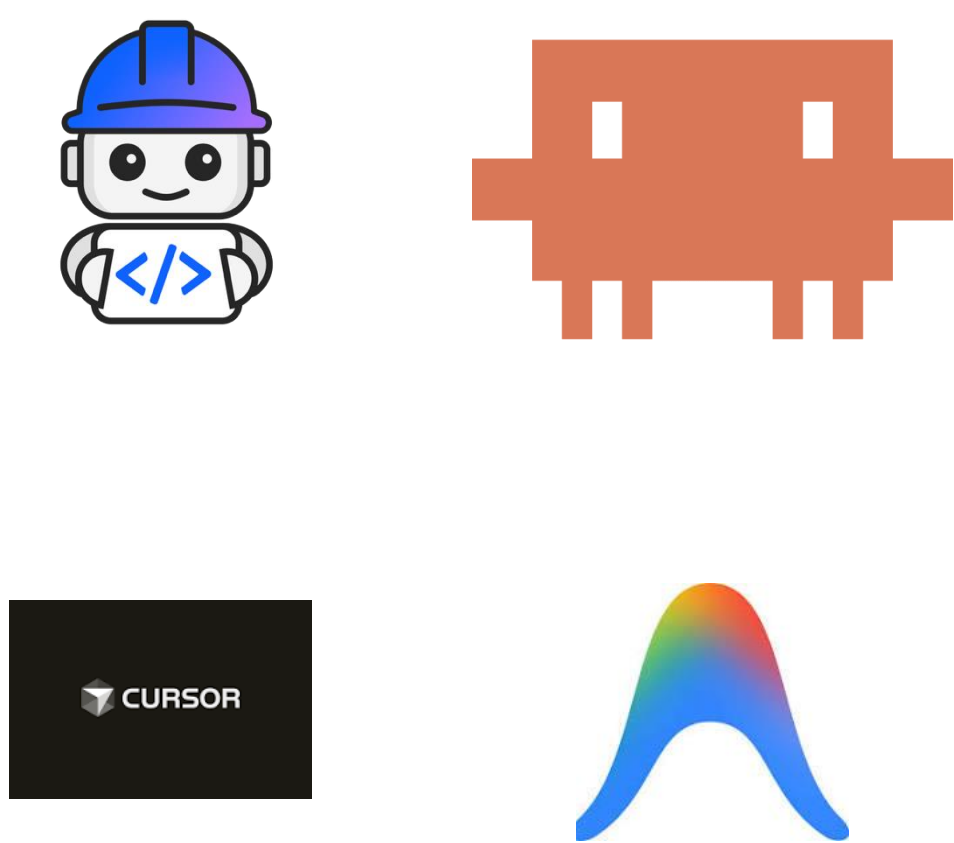


47% of developers ship AI-written code daily



[Source: stackoverflow](https://survey.stackoverflow.co/2025)

Not one major AI code generator tells you the carbon or energy cost



EXPLORER

NOTES-APP

backend

frontend

main.py

1 import python

2

3 Generate a microservices deployment script for Kubernetes:

4 ai = microservices.cast()

5 return microservices

6

7

8

9

10

11

12 def andin_tos():

13 cack_oscript:

14 depiscent:

15 data:

16 "user"

17 events: po

18 ggsometext:

19 #naulty se

20 target "to

21 return los

22

23

OUTLINE

TIMELINE

46% of developers use AI code.

NOBODY is tracking its energy cost.

src > main.py

1 import python

2

3 Generate a microservices deployment script for Kubernetes:

4 ai = microservices.cast()

5 return microservices

6

7

8

9

10

11

12 def andin_tos():

13 cack_oscript:

14 depiscent:

15 data:

16 "user"

17 events: po

18 ggsometext:

19 #naulty se

20 target "to

21 return los

22

23

Suggestion Accepted

ESTIMATED USAGE COSTS

CARBON/ENERGY COST

NOT MEASURED - DATA UNAVAILABLE

?

API TOKEN COST

\$0.04

COMPUTE TIME

0.8 SECONDS

DEVELOPER EFFORT SAVED

45 MINUTES

IBM BOB

Create a ... 22.3k / 270.0k 0.339

Create React frontend 2/5

Create a notes app using React and FastAPI. It should be able to add and delete notes and persist data in a sqllitedb

Let me first check what's already in the workspace.

Listed files in . >

Clean slate. I'll build everything from scratch — FastAPI backend + React frontend.

Edited 2 files : 4s >

Wrote file backend/main.py >

Wrote file backend/requirements.txt >

Now scaffold the React frontend:

Ran npm create vite@latest >

Now write all the React source files:

Edited 2 files : 1ms >

Wrote file frontend/src/index.css >

Wrote file frontend/src/main.jsx >

Now install the Python backend dependencies and validate both sides build cleanly:

One moment...

9 files changed Undo all Show all >

Follow up, ↩ to steer, ⌘ ↩ to queue

+ Agent > Permissions >

Three Crises, One Blind Spot

1

Crisis 1 Legacy debt :

The mountain enterprises already own. Years of accumulated inefficient code - running in production, burning compute and carbon every second, invisible to any ESG dashboard. Nobody rewrote it because it works. It just works expensively.

2

Crisis 2 AI Generation: The fire hose of new inefficiency. AI code generators ship functionally correct but carbon-blind code at unprecedented velocity. 46% of developers use them daily (source- github); ~26-30% of AI code suggestions are accepted by Copilot users on average (source- github copilot impact study); but No tool among major AI code generators currently surface carbon or energy cost estimates to developers.

3

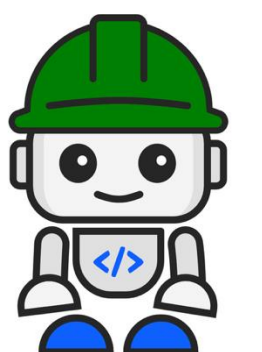
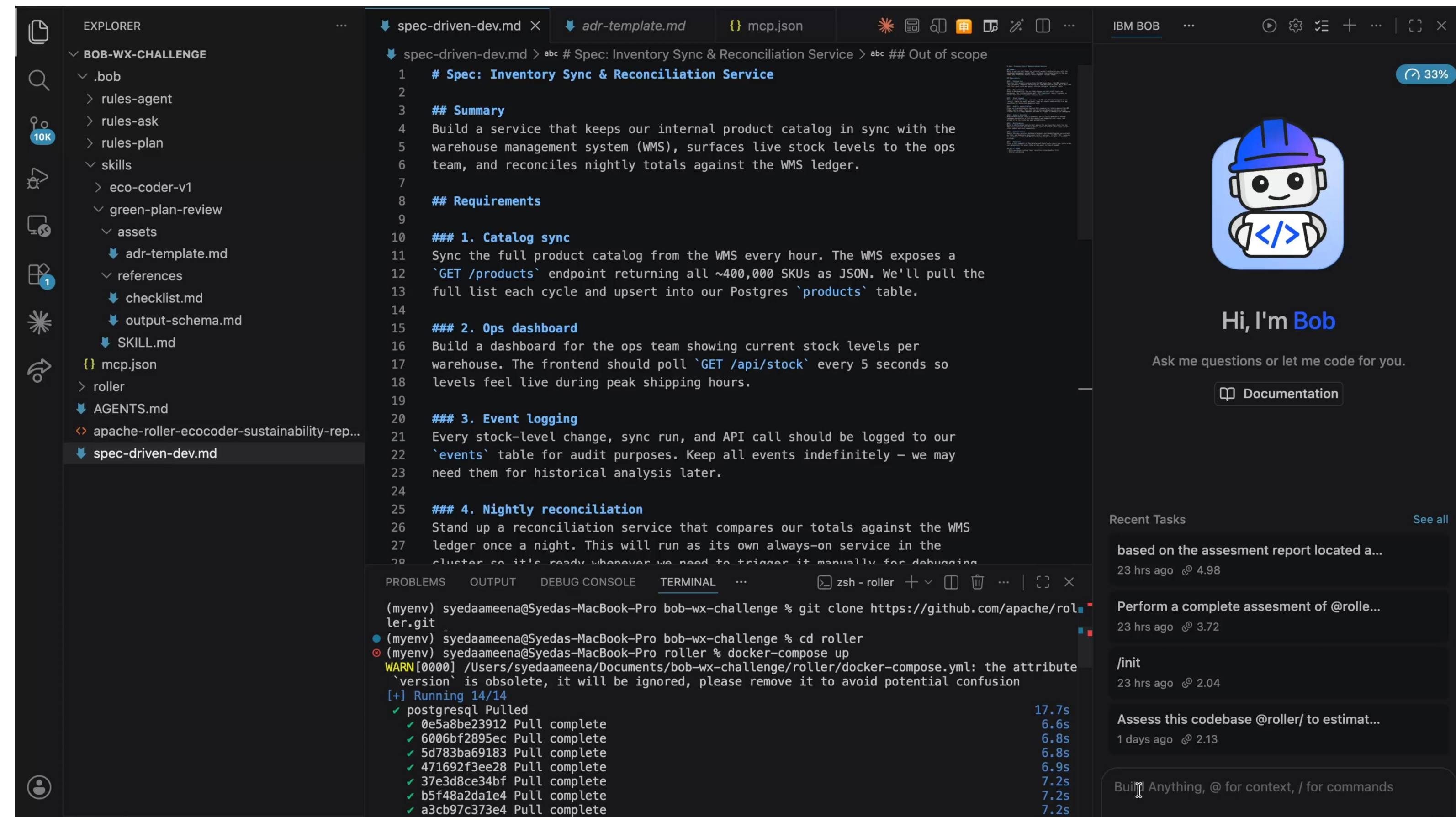
Crisis 3 Infra Ceiling: The layer that cannot reach the code. For example, **IBM Turbonomic continuously right-sizes infrastructure. But it cannot open a source file.** It resizes around inefficiency — it cannot eliminate the root cause inside the application logic.

How EcoCoder Works

Two Moments, One Loop

1. Green Plan Review — Catch Waste Before It's Code

- Catches waste before it's code — reviews specs in plan mode, not after ship
- Shows its math — real ratios, not invented carbon numbers
- Turns decisions into a record — every choice becomes an Architecture Decision Record (ADR)



EcoCoder Fix Report — Apache Roller 6.1.5

P1 green improvements applied · OPT-01 through OPT-04 · Java 11 / Tomcat 9 / EclipseLink JPA

4 OPTs Applied

0 Reverted

26 / 26 Tests Green

Tier-2 estimates · Predicted until OCP Monitor confirms

✓ 26 / 26 tests passed — BannedwordslistTest (5), WeblogTest (2), WeblogEntryTest (16), UserTest (3)

Before vs. Predicted After

MEMORY (AVG_MEM_GB) 0.850 GB Baseline (estimated Tier-2)	MEMORY — PREDICTED AFTER 0.47–0.69 GB –19% to –45%
CPU (AVG VCPU) 0.180 cores Baseline (estimated Tier-2)	CPU — PREDICTED AFTER 0.141–0.161 cores –11% to –22%
ENERGY (KWH/MONTH) 2.424 kWh Baseline	ENERGY — PREDICTED AFTER 1.83–2.14 kWh –12% to –25%
CO₂E (G/MONTH) 793 g Baseline	CO₂E — PREDICTED AFTER 597–700 g –12% to –25%

Relative Improvement (bar = % of baseline)



Predicted Score Improvements

SCORE	BEFORE	PREDICTED AFTER	DRIVER
Memory Efficiency	44 / 100	55–65 / 100	OPT-01, OPT-03, OPT-04 address 3 of 4 major deductions
CPU Efficiency	78 / 100	82–86 / 100	OPT-01 removes M5/P1 hot-path penalty
Ops Posture	40 / 100	58–62 / 100	OPT-03 removes the P9 over-provisioning deduction
Sustainability Score	53 / 100	65–72 / 100	0.5×mem + 0.3×cpu + 0.2×ops

Verification Plan (post-deploy)

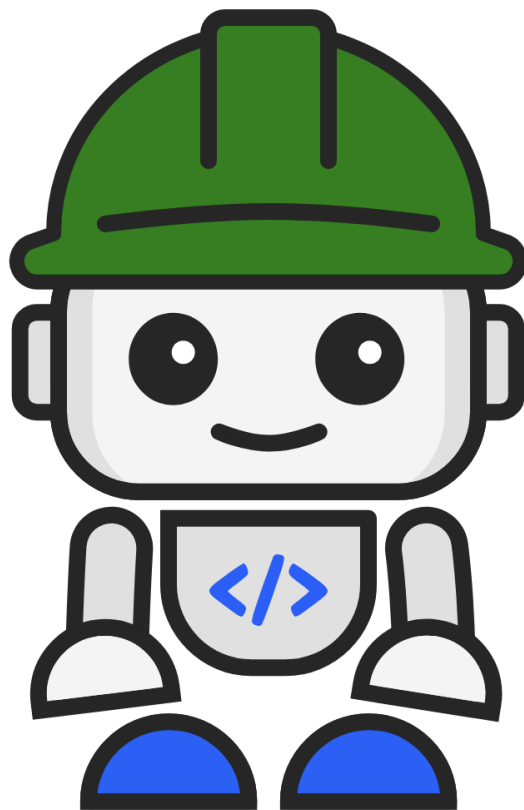
- Deploy the updated container image.
- Wait ≥7 days for stable rolling averages.
- Connect OCP Monitor MCP; call get_avg_cpu, get_avg_mem, get_carbon_footprint for the same 7-day window.
- Re-run CCF formulas on measured values; compare against predicted ranges above.
- If avg_mem_GB > 0.72 GB, heap-dump and inspect entryAnchorToIdMap and EclipseLink L2 cache as primary suspects.
- If sustained heap > 90% at peak, raise –Xmx to 512m and mem_limit to 640m.

All numbers labeled **predicted** (Tier-2 static analysis). Upgrade to **measured** after connecting OCP Monitor MCP and running ≥7-day post-deploy window.

Remaining Opportunities (not applied)

OPT	ISSUE	PRIORITY	RISK	COMPLEXITY
OPT-05	removeMatchingComments: N JPA removes → single bulk DELETE	P2	Medium	M (1–3 days)
OPT-06	updateThumbnail: full BufferedImage decode → streaming thumbnail	P2	Medium	M (1–2 days)
OPT-07	HitCountQueue: fixed-sleep poll → adaptive backoff when queue empty	P2	Low	M (1–2 days)
OPT-08	Add Micrometer/JMX metrics for cache hit rates and memory visibility	P3	Low	L (1+ week)

The Time Cost, Carbon Footprint Before and After



Factors	Before	After	% Gain
Carbon & Cost Visibility	Devs write code blind to cost — waste surfaces only on the cloud bill, months later	Inline gCO ₂ e + USD shown in the IDE at the moment of authorship	100% of new commits gain visibility at point of authorship
Legacy Debt & ESG Compliance	Infra resizes around bad code; ESG teams manually assemble proxy-benchmark evidence	Fixes the application logic itself; auditable, production-data-based ESG output	32%+ cloud waste addressable at the root-cause layer
Efficiency Gains	Inefficiencies compound silently; AI ships new waste at machine speed; nothing quantifies the fix	Scan → Measure → Advise → Refactor → Scorecard, with measured results	29.5–30.6% carbon cut from a single fix (ICSE-SEIP 2026)

The difference between a decision anyone can quantify — and one that never gets made

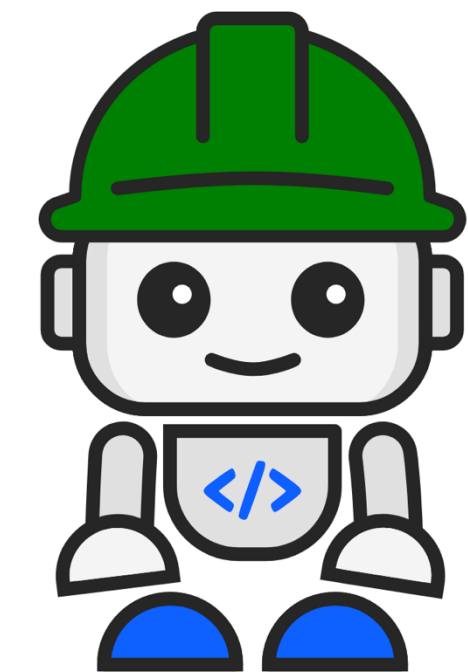
Why This Matters — and What's Next

- **IBM Concert integration** — plug in as Concert's sustainability lens alongside Concert Secure Coder, becoming a fifth pillar next to Operate, Protect, and Resilience
- **Rolling ESG scorecard** — a running, auditable record for leadership — not just per-incident wins, but a number the company can actually report
- **Beyond Turbonomic** — extend the same correlation loop to other infra signal sources (Prometheus, cloud-native autoscalers, other observability platforms), so the pattern isn't locked to one vendor
- **Fleet-wide prioritization** — today it's one repo, one alert at a time; next is ranking waste across an entire application portfolio, so leadership sees the ten repos worth fixing first, not just the one that happened to page someone

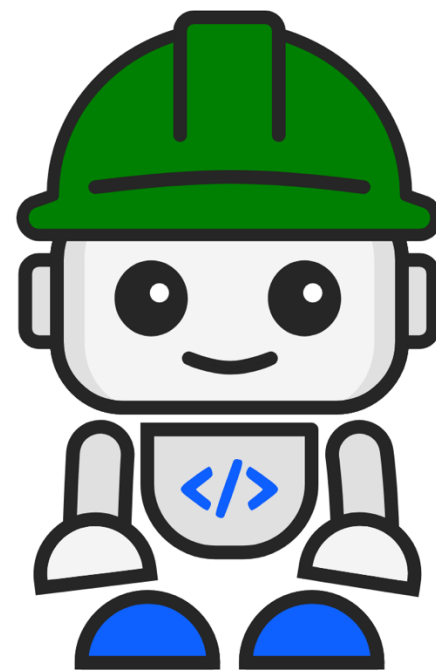
IBM Concert



IBM Turbonomic



Thank you



Not one major AI code generator tells you the carbon or energy cost

